Copel demo — jornada multicanal

AEM headless, Adobe Commerce e Adobe Target: especificação, instruções para o app e roteiro de demo

Date: 23 de outubro de 2026

Adobe

Table of Contents

1. A história da demo 3
2. Status atual 3
3. AEM headless — o que criar 3
4. Como o site usa o banner (já implementado) 8
5. Commerce (sandbox) — referência rápida para o app 9
6. Personalização com Adobe Target 9
7. Instruções para o app (resumo para a IA do app) 10
8. Roteiro da demo (10 a 12 minutos) 11
9. O que eu preciso para fechar 11

Atualizado em 23/10/2026. Substitui a versão anterior deste documento.

1. A história da demo

Um conteúdo, uma conta, uma decisão, em todos os canais.

Peça	Papel na demo	Onde
AEM Sites as a Cloud Service (headless)	Fonte única do **banner da home**: Content Fragments `banner` entregues por GraphQL (persisted queries)	Author `https://author-p68716-e759339.adobeaemcloud.com` · Publish `https://publish-p68716-e759339.adobeaemcloud.com`
Site Copel (AEM Edge Delivery Services + Document Authoring)	Canal web. O bloco `carousel-hero` consome os mesmos Content Fragments	`https://main--copel--denisadobe.aem.page` (preview) / `.aem.live` (live)
App Copel (Vercel)	Canal app. Consome os mesmos Content Fragments e o mesmo Commerce	projeto do app
Adobe Commerce as a Cloud Service	Venda do Seguro Protege Casa: catálogo, conta do cliente, checkout e pedidos	sandbox `na1-sandbox` (seção 5)
Adobe Target (via Web SDK / Edge Network)	Decide **qual banner** cada pessoa vê, no site e no app, com base no que ela já comprou	a configurar (seção 6)

Só o banner da home é headless. As notícias continuam autoradas no site.

2. Status atual

Item	Status
Site migrado (home, falta de luz, notícias, seguro, componentes, loja)	☒ Código na `main`, conteúdo no DA, **preview** gerado. **Live ainda não publicado**
Loja (login, cadastro, conta, checkout expresso, pedidos)	☒ Funcionando no preview. Pedido de teste **000000017** feito em `main--copel--denisadobe.aem.page`
Produtos no Commerce	☒ `protege-casa-basico` (24.90) e `protege-casa-premium` (39.90), pagamento "Cobrança na Fatura" (checkmo)
Bloco do banner no site lendo Content Fragments	☒ Pronto. Lê a persisted query e cai nos slides autorados se o AEM falhar. O gancho de Target fica inativo até ser configurado
Modelo `banner`, Content Fragments e persisted queries no AEM	⏳ **A criar** (seção 3; criação via MCP)
Acesso do site ao AEM sem CORS no AEM	⏳ Rota-ponte no app da Vercel (seção 3.6), sem deploy no AEM
Target (datastream, atividade, audiências, ofertas)	⏳ A configurar (seção 6)

3. AEM headless — o que criar

3.1 Configuração

- Configuração: **`/conf/copel`** (título "Copel"), com **Content Fragment Models** e **GraphQL Persisted Queries** habilitados.

- Endpoint GraphQL: criar o endpoint da configuração `copel` (Tools → General → GraphQL), caminho `/content/cq:graphql/copel/endpoint`.
- Pasta de assets: **`/content/dam/copel`**, com a configuração `/conf/copel` aplicada (Properties → Cloud Configuration) e as subpastas:
  - `/content/dam/copel/banners` (Content Fragments)
  - `/content/dam/copel/imagens/banners` (imagens)

3.2 Modelo de Content Fragment **banner**

Título "Banner", nome técnico **`banner`** (o tipo GraphQL fica `BannerModel`, e as consultas `bannerList` / `bannerByPath`). Os nomes de propriedade são exatamente estes, porque o bloco do site e o app leem esses nomes.

#	Rótulo	Nome (propriedade)	Tipo	Obrigatório	Regras
1	Título	`titulo`	Single line text	sim	máx. 90 caracteres
2	Resumo	`resumo`	Multi line text, **Plain text**	não	máx. 200 caracteres
3	Imagem (desktop)	`imagem`	Content Reference (somente imagem)	sim	raiz `/content/dam/copel/imagens/banners`, ~1600×900
4	Imagem (mobile)	`imagemMobile`	Content Reference (somente imagem)	não	vertical, ~750×1000
5	Texto alternativo	`imagemAlt`	Single line text	não	acessibilidade
6	Botão principal — texto	`ctaTexto`	Single line text	não	
7	Botão principal — link	`ctaLink`	Single line text	não	URL absoluta ou caminho do site (`/seguro-protege-casa`)
8	Botão secundário — texto	`ctaSecundarioTexto`	Single line text	não	
9	Botão secundário — link	`ctaSecundarioLink`	Single line text	não	
10	Canais	`canais`	Enumeration, **múltipla**, checkboxes	sim	opções `site`, `app`
11	Público	`publico`	Enumeration, simples, dropdown	sim	opções `todos`, `visitante`, `cliente-protege-casa` (padrão `todos`)
12	Ordem	`ordem`	Number (Integer)	não	menor aparece primeiro
13	Ativo de	`ativoDe`	Date and Time	não	
14	Ativo até	`ativoAte`	Date and Time	não	
15	SKU relacionado	`skuRelacionado`	Single line text	não	liga o banner a um produto do Commerce

Habilite o modelo depois de salvar (Enable).

3.3 Content Fragments de exemplo (em `/content/dam/copel/banners`)

Suba antes as imagens para `/content/dam/copel/imagens/banners` a partir das URLs abaixo, que são as mesmas usadas hoje no site.

Nome do CF	titulo	resumo	ctaTexto → ctaLink	ctaSecundario	canais	publico	ordem	skuRelacionado	imagem / imagemMobile (origem)
app-copel	Acesse o APP da Copel	Com o APP da Copel, você consegue pagar suas faturas e acessar os serviços do site na palma da sua mão	App Store → https://apple.co/3AUOVJb	Google play → https://play.google.com/store/apps/details?id=com.copel.mbf	site	todos	3	—	…/2026/06/banner-home-desktop-copiar.webp / …/2026/06/banner-home-mobile-copiar-scaled.webp
protege-casa	Segurança e proteção na sua conta de luz!	Contrate o Seguro Protege Casa a partir de R$ 24,90 por mês, pago direto na fatura.	Contratar Protege Casa → /seguro-protege-casa	—	site, app	visitante	1	protege-casa-basico	…/2025/05/banner-site-desktop-2.jpg / …/2025/05/banner-site-mobile-2.jpg
protege-vida-upsell	Sua casa está protegida. E a sua família?	Quem já tem o Protege Casa pode contratar o Protege Vida com o mesmo pagamento na conta de luz.	Conhecer o Protege Vida → https://www.copel.com/site/seguro-protege-vida/	—	site, app	cliente-protege-casa	1	—	…/2025/05/banner-site-desktop-2.jpg / …/2025/05/banner-site-mobile-2.jpg (ou outra imagem de família)
copel-investe	O Paraná cresce. A Copel investe.	Mais qualidade em distribuição, transmissão e geração de energia todos os dias	Saiba mais → https://www.copel.com/site/investe/	—	site, app	todos	4	—	…/2026/09/banner-site-desktop.webp / …/2026/09/banner-site-mobile.webp

Prefixo das imagens (…): `https://www.copel.com/site/wp-content/uploads`. Publique os CFs e as imagens (Publish → Publish incl. referências).

3.4 Persisted queries (configuração `copel`)

`copel/banners-por-canal` — banners de um canal e de um público. O bloco do site usa `canal=site`, e o app usa `canal=app`.

```
query BannersPorCanal($canal: String!, $publico: String = "todos") {
  bannerList(
    filter: {
      canais: { _expressions: [{ value: $canal, _operator: CONTAINS }] }
      publico: { _logOp: OR, _expressions: [{ value: $publico }, { value: "todos" }] }
    }
    sort: "ordem ASC"
  ) {
    items {
      _path
      titulo
      resumo { plaintext }
      imagem { ... on ImageRef { _path _publishUrl _dynamicUrl width height } }
      imagemMobile { ... on ImageRef { _path _publishUrl _dynamicUrl width height } }
      imagemAlt
      ctaTexto ctaLink ctaSecundarioTexto ctaSecundarioLink
      canais publico ordem ativoDe ativoAte skuRelacionado
    }
  }
}
```

`copel/banner-por-path` — um banner específico, usado quando o Target escolhe o banner.

```
query BannerPorPath($path: String!) {
  bannerByPath(_path: $path) {
    item {
      _path
      titulo
      resumo { plaintext }
      imagem { ... on ImageRef { _path _publishUrl _dynamicUrl width height } }
      imagemMobile { ... on ImageRef { _path _publishUrl _dynamicUrl width height } }
      imagemAlt
      ctaTexto ctaLink ctaSecundarioTexto ctaSecundarioLink
      canais publico ordem skuRelacionado
    }
  }
}
```

Chamadas (GET, cacheáveis na CDN do Publish):

```
https://publish-p68716-e759339.adobeaemcloud.com/graphql/execute.json/copel/banners-por-canal;canal=site
https://publish-p68716-e759339.adobeaemcloud.com/graphql/execute.json/copel/banners-por-canal;canal=app;publico=cliente-protege-casa
https://publish-p68716-e759339.adobeaemcloud.com/graphql/execute.json/copel/banner-por-path;path=/content/dam/copel/banners/protege-vida-upsell
```

Depois de salvar, publique as persisted queries.

3.5 Passo a passo simplificado (sem deploy, só pela interface ou pelo MCP)

Nada aqui exige código, Cloud Manager ou deploy. São 8 passos, cerca de 30 minutos.

1. **Configuração.** Tools → General → **Configuration Browser** → Create:
   - título "Copel", nome `copel`;
   - marque **Content Fragment Models** e **GraphQL Persisted Queries**.
2. **Modelo.** Tools → General → **Content Fragment Models** → pasta *Copel* → Create "Banner":

- adicione os campos da tabela 3.2, com os mesmos nomes de propriedade;
- salve e clique em **Enable**.
- Campos **essenciais**: `titulo`, `resumo`, `imagem`, `ctaTexto`, `ctaLink`, `canais`, `publico`, `ordem`.
- Os outros são opcionais, e o site funciona sem eles.

3. **Pastas.** Assets → Files:
   - crie a pasta `copel`;
   - em Properties → **Cloud Services**, escolha a configuração `/conf/copel`;
   - dentro dela, crie `banners` e `imagens`.
4. **Imagens.** Suba em `copel/imagens` as imagens da tabela 3.3.
5. **Content Fragments.** Em `copel/banners`: Create → Content Fragment → modelo *Banner* → crie os 4 da tabela 3.3.
6. **Persisted query** (também sem deploy, pela interface):
   - Tools → General → **GraphQL** → Create endpoint para `copel`.
   - Tools → General → **GraphQL Query Editor**, endpoint `copel`.
   - Cole a query `banners-por-canal` (3.4) → **Save As** `banners-por-canal`.
   - Repita com `banner-por-path`.
7. **Publicar.** Selecione as pastas `copel/banners` e `copel/imagens` → **Quick Publish**, incluindo as referências. No Query Editor, publique as 2 queries.
8. **Testar** no navegador: `https://publish-p68716-e759339.adobeaemcloud.com/graphql/execute.json/copel/banners-por-canal;canal=site` precisa devolver `data.bannerList.items` com os banners.

> **Atalho:** se o MCP não criar persisted queries, pule o passo 6. O proxy do app (3.6) pode montar a consulta GraphQL e enviar por POST para `…/content/cq:graphql/copel/endpoint.json`. Para isso, só o endpoint GraphQL precisa existir.

3.6 Sem CORS no AEM: o app serve de ponte

A configuração de CORS no AEM exigiria deploy, por isso **não vamos mexer no AEM**: - **App (Vercel):** busca os banners pelo servidor (Server Component ou Route Handler). Chamada de servidor não precisa de CORS. - **Site (EDS):** chama uma rota do próprio app, que repassa ao AEM e devolve a resposta com o cabeçalho CORS. O bloco do banner aceita qualquer URL na linha "Fonte".

Rota para o app (Next.js App Router), em `app/api/aem/[...query]/route.ts`:

```
// Ponte de leitura para as persisted queries do AEM Publish, com CORS para o site Copel.
const AEM = 'https://publish-p68716-e759339.adobeaemcloud.com';
const QUERIES = new Set(['copel/banners-por-canal', 'copel/banner-por-path']);
const ORIGINS = [
  /^https:\/\/[a-z0-9-]+--copel--denisadobe\.aem\.(page|live)$/,
  /^http:\/\/localhost:3000$/,
];

function cors(origin: string | null): Record<string, string> {
  const ok = origin && ORIGINS.some((re) => re.test(origin));
  return ok ? { 'Access-Control-Allow-Origin': origin!, Vary: 'Origin' } : {};
}

export async function GET(req: Request, { params }: { params: Promise<{ query: string[] }
> }) {
  const { query } = await params;
  // ex.: /api/aem/copel/banners-por-canal;canal=site
  const [name, ...vars] = decodeURIComponent(query.join('/')).split(';');
```

```
  if (!QUERIES.has(name)) return new Response('Not found', { status: 404 });
  const url = `${AEM}/graphql/execute.json/${name}${vars.length ? `;${vars.join(';')}` :
''}`;
  const res = await fetch(url, { next: { revalidate: 60 } });
  return new Response(await res.text(), {
    status: res.status,
    headers: {
      'Content-Type': 'application/json',
      'Cache-Control': 'public, s-maxage=60, stale-while-revalidate=300',
      ...cors(req.headers.get('origin')),
    },
  });
}

export function OPTIONS(req: Request) {
  return new Response(null, {
    status: 204,
    headers: { ...cors(req.headers.get('origin')), 'Access-Control-Allow-Methods': 'GET,
OPTIONS' },
  });
}
```

No site, a linha "Fonte" do banner da home passa a apontar para a ponte: `https://<app>.vercel.app/api/aem/copel/banners-por-canal;canal=site`

O app usa a mesma rota ou chama o AEM direto pelo servidor.

Imagens: as tags `<img>` carregam direto do Publish (`_publishUrl`), sem precisar de CORS.

4. Como o site usa o banner (já implementado)

Bloco `carousel-hero` (home). No Document Authoring, a primeira linha da tabela do bloco vira a configuração:

Carousel Hero	
Fonte	`/graphql/execute.json/copel/banners-por-canal;canal=site`
Escopo Target	`copel-home-hero` *(opcional)*
(slides autorados abaixo = fallback)	

- **Endereço do Publish:** um caminho relativo usa `aem-publish-url` do `config.json` (`https://publish-p68716-e759339.adobeaemcloud.com`).
- **Ordem de carregamento:** os slides autorados aparecem na hora. Os do AEM entram quando chegam, com timeout de cerca de 4 s, então o LCP não espera o AEM.
- **Fallback:** se der erro, timeout ou vier lista vazia, ficam os slides autorados.
- **Atributos:** cada slide vindo do AEM leva `data-cf-path` (o `_path` do CF) e o bloco recebe `data-source="content-fragments"`.
- **Target:** com "Escopo Target" e Web SDK (`window.alloy`) na página, o bloco pede a decisão. Se a resposta trouxer `{ "bannerPath": "…" }`, esse banner vai para o primeiro slide (ou é buscado por `banner-por-path`) e a exibição é registrada no Target.

Para ligar na home: depois que os CFs existirem, basta adicionar a linha "Fonte" no `index` do DA e gerar o preview. Hoje a home continua com os slides autorados.

5. Commerce (sandbox) — referência rápida para o app

- **GraphQL:** `https://na1-sandbox.api.commerce.adobe.com/TcvPqpzQp1aRq3ohMQt2ct/graphql`
- **Cabeçalhos:**
  - `Content-Type: application/json`
  - `Store: default`
  - `Magento-Store-Code: main_website_store`
  - `Magento-Store-View-Code: default`
  - `Magento-Website-Code: base`
  - Não use `x-api-key`.
- **SKUs:** `protege-casa-basico` (24.90) e `protege-casa-premium` (39.90).
  - Produtos virtuais: sem entrega.
  - Os preços vêm em USD; site e app **exibem como R$** (ajuste de demo: trocar a moeda na exibição).
- **Fluxo (o mesmo do site):**
  1. `generateCustomerToken` → `Authorization: Bearer <token>`. Cadastro: `createCustomerV2`.
  2. `customerCart { id }` → remover itens existentes → `addProductsToCart(cartId, [{ sku, quantity: 1 }])`.
  3. `setBillingAddressOnCart`: obrigatório mesmo para produto virtual. Use o endereço do cliente ou o neutro: Rua José Izidoro Biazetto, 158 · Curitiba · PR · 81200-240 · BR · tel. 4133310000.
  4. `setPaymentMethodOnCart(code: "checkmo")`: o título é "Cobrança na Fatura".
  5. `placeOrder(input: { cart_id })` → `orderV2.number`.
  6. Pedidos: `customer { orders { items { number status order_date total { grand_total { value } } items { product_sku product_name } } } }`.
- **Mesma conta e mesmos pedidos no site e no app.** O site guarda a sessão no cookie `auth_dropin_user_token`; o app guarda o próprio token.
- Nomes de cliente com `+` ou símbolos são recusados pelo Commerce: limpe antes de enviar.

6. Personalização com Adobe Target

6.1 Ideia

O Target decide **qual Content Fragment de banner** cada pessoa vê. A oferta do Target **não guarda conteúdo**, só aponta para o CF:

```
{ "bannerPath": "/content/dam/copel/banners/protege-vida-upsell" }
```

O conteúdo é editado uma vez no AEM, a decisão é tomada uma vez no Target, e o site e o app renderizam do mesmo jeito.

6.2 O que configurar

1. **Datastream** (Data Collection → Datastreams) "copel-demo", com **Adobe Target** habilitado. Opcional: Adobe Analytics / AEP.
2. **Web SDK no site:** carregar o Alloy com `orgId` e `datastreamId` no carregamento atrasado da página. Recomendado: o plugin `aem-martech` do Edge Delivery. O bloco do banner já chama `alloy('sendEvent', { decisionScopes: ['copel-home-hero'] })` quando `window.alloy` existe. **Preciso de `orgId` e `datastreamId` para ligar.**
3. **No app:** Web SDK no navegador, ou **Edge Network Server API** no servidor Next.js (`POST https://server.adobedc.net/ee/v2/interact?dataStreamId=…`), com o mesmo `decisionScopes: ["copel-home-hero"]`.

4. **Identidade entre canais:** depois do login no Commerce, envie no `identityMap` um **CRM ID** (por exemplo o e-mail do cliente em hash SHA-256, ou o `customer.id`) com `authenticatedState: "authenticated"`. Com isso a mesma pessoa é reconhecida no site e no app.
5. **Sinal de compra:** depois do `placeOrder`, envie um evento com:
   - `xdm.eventType: "commerce.purchases"`
   - `productListItems: [{ SKU: "protege-casa-basico", priceTotal: 24.9 }]`
   - `data.__adobe.target.profile: { "protegeCasa": "true" }`

   O Target guarda `user.protegeCasa = true` no perfil.
6. **Atividade no Target:** do tipo **Experience Targeting**, form-based, location/escopo `copel-home-hero`.
   - Audiência "Cliente Protege Casa" (`profile.protegeCasa = true`) → oferta JSON `{ "bannerPath": "/content/dam/copel/banners/protege-vida-upsell" }`.
   - Todos os outros visitantes → oferta JSON `{ "bannerPath": "/content/dam/copel/banners/protege-casa" }`.
   - Métrica: clique no CTA ou o evento `commerce.purchases`.

6.3 Plano B sem Target

Se o Target não ficar pronto a tempo, a mesma história funciona com o campo `publico` do CF. O app (e o site, pelo `;publico=` da query) pede `publico=cliente-protege-casa` quando o cliente logado já tem um pedido com SKU `protege-casa-*`, e `publico=visitante` nos outros casos.

---

7. Instruções para o app (resumo para a IA do app)

1. **Banner da home:**
   - Busque no servidor (Next.js Server Component ou Route Handler) `…/execute.json/copel/banners-por-canal;canal=app;publico=<publico>`, com `revalidate` de 60 s.
   - Use as imagens de `_dynamicUrl` (prefixe o host do Publish e use `?width=…&preferwebp=true`) ou de `_publishUrl`.
   - Ordene por `ordem` e respeite `ativoDe` e `ativoAte`.
2. **Personalização:**
   - Antes de renderizar o banner, pergunte ao Target pela Edge Network API: escopo `copel-home-hero`, mais o `identityMap` com o CRM ID se o usuário estiver logado.
   - Se vier `bannerPath`, busque `banner-por-path` e coloque esse banner em primeiro.
   - Registre a exibição: evento `decisioning.propositionDisplay`.
   - Sem resposta do Target, use o plano B (campo `publico`).
3. **Commerce:**
   - Login, cadastro, compra e "Meus pedidos" exatamente como na seção 5.
   - Depois da compra, envie o evento de compra (seção 6.2, item 5) para o próximo banner ser o de upsell.
4. **Visual:**
   - Cores: laranja `#F5821E` nas barras e na marca; botões `#C85000` com texto branco; títulos `#E06000`; grafite `#373A3C`; texto `#151C21`.
   - Fontes: Quenda nos títulos (peso 500, `https://www.copel.com/site/wp-content/uploads/2021/12/Quenda-Medium.ttf`) e Univers LT 55 Roman no texto (`https://www.copel.com/site/wp-content/uploads/2021/12/Univers-LT-55-Roman.ttf`), ambas com CORS liberado. Use `font-synthesis: none`.
5. **Segredos:** nenhum segredo no app para essas leituras (persisted queries publicadas e Commerce storefront são públicos). Tokens de cliente só na sessão do usuário.

8. Roteiro da demo (10 a 12 minutos)

#	Tela	Fala / ação	O que demonstra
1	Site (home) e app (home), lado a lado, como visitante	"Mesmo banner nos dois canais: Protege Casa."	AEM headless: um CF, dois canais
2	AEM Author → CF `protege-casa`	Editar o título ("…a partir de R$ 24,90") → Publish	Autoria única
3	Recarregar o site e o app	Os dois mostram o texto novo	Entrega headless e cache
4	Site → Seguro Protege Casa → "Quero o Plano Básico"	Login (Commerce) → checkout expresso → **Cobrança na Fatura** → pedido confirmado	Commerce no EDS, venda de serviço sem entrega
5	App → Meus pedidos	O mesmo pedido aparece	Uma conta, um histórico
6	App (logado) e site (logado) → home	O banner agora é **"Sua casa está protegida. E a sua família?"** (Protege Vida)	Target: decisão por perfil, igual nos dois canais
7	Target → relatório da atividade	Exibições e conversões por experiência	Medição
8	Fechamento	"Conteúdo no AEM, transação no Commerce, decisão no Target, experiência em qualquer canal."	Visão de plataforma

Dicas de preparo: - Crie antes um cliente "Ana" no Commerce. - Use uma janela anônima para a fase visitante. - Deixe uma segunda conta, ainda sem pedido, para repetir a demo. - Se o Target não estiver pronto, use o plano B (6.3), que mostra a mesma mudança de banner.

9. O que eu preciso para fechar

1. **AEM:** criar o modelo, os CFs e as queries pelo passo a passo 3.5 (interface ou MCP, **sem deploy**). Os nomes de propriedade da tabela 3.2 não podem mudar.
2. **Ponte no app:** publicar a rota `/api/aem/...` da seção 3.6 no app da Vercel e me passar o domínio do app.
3. **Target / Web SDK:** `orgId`, `datastreamId` e o workspace/propriedade do Target. Com isso eu ligo o Alloy no site e escrevo a atividade.
4. **Publicar o site no live:** o pedido precisa vir numa mensagem própria, em inglês, por exemplo: *"Please publish all Copel pages to live (admin.hlx.page)."* A checagem de segurança da plataforma não aceitou o pedido em português.